

# Variables

---

Las variables se utilizan para almacenar datos que pueden referenciarse y manipularse durante la ejecución del programa.

---

Las variables se utilizan para almacenar datos que pueden referenciarse y manipularse durante la ejecución del programa.

No requieren una declaración explícita de tipo. El tipo de la variable se infiere a partir del valor asignado.

```
1  x = 10  
2  x = "Now a string"
```

# Reglas para nombrar variables

---

- Los nombres de variables solo pueden contener letras, dígitos y guiones bajos (\_).
- Un nombre de variable no puede empezar con un dígito.
- Los nombres de variable distinguen entre mayúsculas y minúsculas (myVar y myvar son diferentes).
- Evite usar palabras clave de Python (por ejemplo, if, else, for) como nombres de variable.

---

## CORRECTO

```
1 age = 21
2 _colour = "lilac"
3 total_score = 90
```

## INCORRECTO

```
1 lname = "Error" # Empieza con un dígito
2 class = 10 # 'class' es una palabra reservada
3 user-name = "Doe" # contiene un guión
```

# Asignación Básica

---



variables.py

```
1 x = 10
2 y = 3.14
3 z = "Hi"
4 x = "Now a string"
```

# Asignación Múltiple

---



variables.py

```
1 a = b = c = 100
2 print(a, b, c)
3
4 x, y, z = 1, 2.5, "Python"
5 print(x, y, z)
6
```

# Conversión de Tipos

---

```
1    s1 = "10"  
2    n = int(s1)  
3    cnt = 5  
4    f = float(cnt)  
5    age = 25  
6    s2 = str(age)  
7  
8    print(type(n))  
9    print(type(f))  
10   print(type(s2))
```

# Ámbito de una variable

---

**Variables locales.** Las variables definidas dentro de una función son locales a dicha función.

```
1 def f():  
2     a = "I am local"  
3     print(a)  
4 f()  
5 print(a) # Esto genera un error  
6
```



---

**Variables globales.** Las variables definidas fuera de cualquier función son globales y se puede acceder a ellas dentro de las funciones mediante la palabra clave `global`.

```
1 a = "I am global"
2
3 def f():
4     global a
5     a = "Modified globally"
6     print(a)
7 f()
8 print(a)
```

# Tipos de datos

---

# Numéricos

---

**Enteros:** Este valor se representa mediante la clase **int**. Contiene números enteros positivos o negativos (sin fracciones ni decimales). En Python, no hay límite para la longitud de un valor entero.

**Float:** Este valor se representa mediante la clase **float**. Es un número real con representación en coma flotante. Se especifica mediante un punto decimal.

**Números complejos:** Un número complejo se representa mediante una clase **complex**. Se especifica como (parte real) + (parte imaginaria) j. Por ejemplo: 2+3j

---

```
a = 5  
print(type(a))
```

```
b = 5.0  
print(type(b))
```

```
c = 2 + 4j  
print(type(c))
```

# Secuencia

---

El tipo de datos de secuencia en Python es la colección ordenada de tipos de datos similares o diferentes. Las secuencias permiten almacenar múltiples valores de forma organizada y eficiente. Existen varios tipos de datos de secuencia en Python:

- String
- Lista
- Tupla

---

**String:** Las cadenas de Python son matrices de bytes que representan caracteres Unicode. En Python, no existe un tipo de dato de carácter; un carácter es una cadena de longitud uno. Se representa mediante la clase **str**.

Las cadenas en Python se pueden crear usando comillas simples, dobles o triples. Se puede acceder a caracteres individuales de una cadena mediante el índice.

---

```
s = "Welcome to the Geeks World"  
print(s)  
  
print(type(s))  
  
print(s[1])  
print(s[2])  
print(s[-1])
```

# Secuencia

---

Lista: Las listas son como matrices, declaradas en otros lenguajes, que constituyen una colección ordenada de datos. Son muy flexibles, ya que los elementos de una lista no necesitan ser del mismo tipo.

```
a = []
```

```
a = [1, 2, 3]  
print(a)
```

```
b = ["Geeks", "For", "Geeks", 4, 5]  
print(b)
```



# Secuencia

---

```
b = ["Geeks", "For", "Geeks", 4, 5]
print("Accessing element from the list")
print(b[0])
print(b[2])

print("Accessing element using negative indexing")
print(b[-1])
print(b[-3])
```

---

**Tupla:** Al igual que una lista, una tupla es una colección ordenada de objetos de Python. La única diferencia entre una tupla y una lista es que las tuplas son inmutables. Una vez creadas, no se pueden modificar.

```
1  tup1 = ()
2  tup2 = ("Geeks", "For")
3  print("\nTuple with the use of String: ", tup2)
```

---

```
1  tup = ([1, 2, 3, 4, 5])
2  print(tup[0])
3  print(tup[-1])
4  print(tup[-3])
```

# Booleano

---

Tipo de dato de Python con uno de los dos valores predefinidos: **True** o **False**. Esto se denota por la clase **bool**.



```
print(type(True))  
print(type(False))  
print(type(true)) # genera un error
```

# Set

---

En los tipos de datos de Python, un **set** es una colección desordenada de tipos de datos, iterable, mutable y sin elementos duplicados. El orden de los elementos de un conjunto no está definido, aunque puede constar de varios elementos.



```
s1 = set()
s1 = set("GeeksForGeeks")
print("Set with the use of String: ", s1)

s2 = set(["Geeks", "For", "Geeks"])
print("Set with the use of String: ", s2)
```

# Set

---



```
set1 = set(["Geeks", "For", "Geeks"])

for i in set1:
    print(i, end=" ")

print("Geeks" in set1)
```

# Diccionario

---

Un diccionario en Python es una colección de valores de datos que se utiliza para almacenarlos como un mapa. A diferencia de otros tipos de datos de Python que solo contienen un valor como elemento, un diccionario contiene un par clave-valor. Cada par clave-valor en un diccionario se separa con dos puntos, mientras que cada clave se separa con una coma.



```
d = {}  
d = {1: "Geeks", 2: "For", 3: "Geeks"}  
print(d)  
  
d1 = dict({1: "Geeks", 2: "For", 3: "Geeks"})  
print(d1)
```

# Diccionario

---



```
d = {1: "Geeks", "name": "For", 3: "Geeks"}
```

```
print(d["name"])
```

```
print(d.get(3))
```



# Operadores

---

Los operadores en general se utilizan para realizar operaciones sobre valores y variables.

# Aritméticos

---

```
a = 15
b = 4

# Suma
print("Addition", a + b)

# Multiplicación
print("Multiplication:", a * b)

# División
print("Division:", a / b)
```

---

# Floor Division

`print("Floor Division:", a // b)`

# Módulo

`print("Modulus:", a % b)`

# Potencia

`print("Exponentiation:", a ** b)`

# Comparación

---



operadores.py

```
a = 13
```

```
b = 33
```

```
print(a > b)
```

```
print(a < b)
```

```
print(a == b)
```

```
print(a != b) # a es diferente de b
```

```
print(a >= b)
```

```
print(a <= b)
```

# Lógicos

---



operadores.py

```
a = True  
b = False
```

```
print(a and b)  
print(a or b)  
print(not a)
```

# Asignación

---

```
a = 10
b = 20

print(b)

b += a # b = b + a
print(b)

b -= a # b = b - a
print(b)

b *= a # b = b * a
print(b)
```

# Identidad

---

En Python, los operadores de identidad **is** y **is not** se utilizan para comprobar si dos valores se encuentran en la misma parte de la memoria. Dos variables iguales no implican que sean idénticas.

```
a = 10
b = 20
c = a

print(a is not b)
print(a is c)
```

# Membresía

---

En Python, **in** y **not in** son los operadores de pertenencia que se utilizan para probar si un valor o una variable está en una secuencia.



---

```
1  x = 24
2  y = 20
3  list = [10, 20, 30, 40, 50]
4
5  if (x not in list):
6      print("x is NOT present in given list")
7  else:
8      print("x is present in given list")
9
10 if (y in list):
11     print("y is present in given list")
12 else:
13     print("y is NOT present in given list")
```

# Operador Ternario

---

Simplemente permiten probar una condición en una sola línea, reemplazando el if-else multilínea, lo que compacta el código.

```
a, b = 10, 20  
min = a if a < b else b  
print(min)
```

# Input

---

# input()

---

La función input() de Python se utiliza para tomar datos del usuario. Por defecto, devuelve la entrada del usuario un string.

```
name = input("Enter your name: ")  
print("Hello,", name, "! Welcome!")
```

---

Tomamos múltiples entradas del usuario en una sola línea, dividiendo los valores ingresados por el usuario en variables independientes para cada valor mediante el método `split()`.

```
x, y = input("Enter two values: ").split()
print("Number of boys: ", x)
print("Number of girls: ", y)
```

---

```
x, y, z = input("Enter three values: ").split()
print("Total number of students: ", x)
print("Number of boys: ", y)
print("Number of girls: ", z)
```

# int()

---

El código solicita al usuario que ingrese un número entero que represente la cantidad de rosas, convierte la entrada a un entero mediante conversión de tipos y luego imprime el valor entero.

```
n = int(input("How many roses?: "))  
print(n)
```

# float()

---

El código solicita al usuario que ingrese el precio de cada rosa como un número de punto flotante, convierte la entrada a un número decimal mediante conversión de tipos y luego imprime el precio.

```
price = float(input("Price of each rose?: "))  
print(price)
```



# Output

---

# print()

---

En esencia, imprimir la salida en Python es sencillo gracias a la función `print()`. Esta función permite mostrar texto, variables y expresiones en la consola.

En este ejemplo, "¡Hello, World!" es una cadena literal entre comillas dobles. Al ejecutarse, esta sentencia mostrará el texto en la consola.

```
s = "Bob"  
print(s)
```

---

Podemos usar la función **print()** para imprimir variables individuales o múltiples. Podemos imprimir varias variables separándolas con comas.

```
s = "Alice"  
age = 25  
city = "New York"  
print(s, age, city)
```

# Formato

---

```
amount = 150.75
print("Amount: ${:.2f}".format(amount))

print("Python", end='@')
print("GeeksForGeeks")

print('G', 'F', 'G', sep='')

print('09', '12', '2016', sep='-')

print('user', '@', 'geeksforgeeks', sep='@')
```